

ThinkBeforeClick — File Roles Reference

Backend architecture: what each file does and how it connects to the pipeline

Entry Point

<code>main.py</code>	Starts the FastAPI app and mounts all routers (analyze, health, report). The single entry point that ties every module together.
----------------------	--

Request Handling

<code>api/analyze.py</code>	Route handler for POST /analyze. Receives the request, calls the validated schema, invokes the services layer, and returns the final JSON verdict.
<code>api/health.py</code>	Route handler for GET /health. Simple uptime check used by Render/monitoring to confirm the server is alive.
<code>api/report.py</code>	Route handler for the report endpoint. Lets users flag or submit feedback on a scan result.

Validation

<code>schemas/analyze_schema.py</code>	Pydantic model that validates the incoming request (message/URL) before it reaches any business logic, and shapes the outgoing response.
--	--

Business Logic (services/)

<code>services/rule_engine.py</code>	Checks the input against regex patterns and known scam keywords. Fast, deterministic first-pass detection.
<code>services/ml_engine.py</code>	Loads classifier.joblib and vectorizer.joblib, converts text to TF-IDF features, and predicts a risk score using Logistic Regression.
<code>services/threat_intel.py</code>	Calls external APIs — Google Safe Browsing and WHOIS — to check if the URL/domain is flagged or newly registered.
<code>services/fusion_engine.py</code>	Combines the three scores (rule 30% / ML 40% / threat-intel 30%) into one final verdict: Safe, Suspicious, or High Risk.

Machine Learning (ml/)

<code>ml/train_model.py</code>	Training script used offline to build and export the classifier and vectorizer used by ml_engine.py.
<code>ml/model/classifier.joblib</code>	Serialized trained Logistic Regression model, loaded at runtime by ml_engine.py.

ml/model/vectorizer.joblib	Serialized TF-IDF vectorizer that converts raw text into the numeric features the classifier expects.
-----------------------------------	---

Persistence (db/)

db/database.py	Sets up the SQLAlchemy engine/session for connecting to SQLite.
-----------------------	---

db/models.py	Defines the database tables (e.g. scan history) — what gets stored per analyzed request.
---------------------	--

Configuration

core/config.py	Loads environment variables (like SAFE_BROWSING_API_KEY) and app-wide settings.
-----------------------	---

core/logging.py	Sets up logging format/level so requests and errors are traceable.
------------------------	--

Clients

webapp/static/js/analyzer.js	Frontend script that sends the URL/message to /analyze and renders the returned verdict (triggers GSAP animations).
-------------------------------------	---

extension/popup/popup.js	Browser extension equivalent of analyzer.js — sends the current page/URL to the backend from the popup UI.
---------------------------------	--